

An Overview of the Agent Architecture

This article covers key concepts in the architecture of the Agent Framework, including foundational principles, design objectives, and strategic goals.

Goals

The `Agent Framework` was developed with the following key priorities in mind:

- The Semantic Kernel agent framework serves as the core foundation for implementing agent functionalities.
- Multiple agents of different types can collaborate within a single conversation, each contributing their unique capabilities, while integrating human input.
- An agent can engage in and manage multiple concurrent conversations simultaneously.

Agent

The abstract `Agent` class serves as the core abstraction for all types of agents, providing a foundational structure that can be extended to create more specialized agents. This base class forms the basis for more specific agent implementations, all of which leverage the Kernel's capabilities to execute their respective functions. See all the available agent types in the [Agent Types](#) section.

The underlying Semantic Kernel `Agent` abstraction can be found [here](#).

Agents can either be invoked directly to perform tasks or be orchestrated by different patterns. This flexible structure allows agents to adapt to various conversational or task-driven scenarios, providing developers with robust tools for building intelligent, multi-agent systems.

Agent Types in Semantic Kernel

- [ChatCompletionAgent](#)
- [OpenAIAssistantAgent](#)
- [AzureAIAgent](#)
- [OpenAIResponsesAgent](#)
- [CopilotStudioAgent](#)

Agent Thread

The abstract `AgentThread` class serves as the core abstraction for threads or conversation state. It abstracts away the different ways in which conversation state may be managed for different agents.

Stateful agent services often store conversation state in the service, and you can interact with it via an id. Other agents may require the entire chat history to be passed to the agent on each invocation, in which case the conversation state is managed locally in the application.

Stateful agents typically only work with a matching `AgentThread` implementation, while other types of agents could work with more than one `AgentThread` type. For example, `AzureAIAgent` requires a matching `AzureAIAgentThread`. This is because the Azure AI Agent service stores conversations in the service, and requires specific service calls to create a thread and update it. If a different agent thread type is used with the `AzureAIAgent`, we fail fast due to an unexpected thread type and raise an exception to alert the caller.

Agent Orchestration

Important

Agent Orchestration features in the Agent Framework are in the experimental stage. They are under active development and may change significantly before advancing to the preview or release candidate stage.

Note

If you have been using the [AgentGroupChat](#) orchestration pattern, please note that it is no longer maintained. We recommend developers to use the new [GroupChatOrchestration](#) pattern. A migration guide is provided [here](#).

The [Agent Orchestration](#) framework in Semantic Kernel enables the coordination of multiple agents to solve complex tasks collaboratively. It provides a flexible structure for defining how agents interact, share information, and delegate responsibilities. The core components and concepts include:

- **Orchestration Patterns:** Pre-built patterns such as Concurrent, Sequential, Handoff, Group Chat, and Magentic allow developers to choose the most suitable collaboration model for their scenario. Each pattern defines a different way for agents to communicate and process tasks (see the [Orchestration patterns](#) table for details).

- **Data Transform Logic:** Input and output transforms allow orchestration flows to adapt data between agents and external systems, supporting both simple and complex data types.
- **Human-in-the-loop:** Some patterns support human-in-the-loop, enabling human agents to participate in the orchestration process. This is particularly useful for scenarios where human judgment or expertise is required.

This architecture empowers developers to build intelligent, multi-agent systems that can tackle real-world problems through collaboration, specialization, and dynamic coordination.

Agent Alignment with Semantic Kernel Features

The `Agent Framework` is built on the foundational concepts and features that many developers have come to know within the Semantic Kernel ecosystem. These core principles serve as the building blocks for the Agent Framework's design. By leveraging the familiar structure and capabilities of the Semantic Kernel, the Agent Framework extends its functionality to enable more advanced, autonomous agent behaviors, while maintaining consistency with the broader Semantic Kernel architecture. This ensures a smooth transition for developers, allowing them to apply their existing knowledge to create intelligent, adaptable agents within the framework.

Plugins and Function Calling

[Plugins](#) are a fundamental aspect of the Semantic Kernel, enabling developers to integrate custom functionalities and extend the capabilities of an AI application. These plugins offer a flexible way to incorporate specialized features or business-specific logic into the core AI workflows. Additionally, agent capabilities within the framework can be significantly enhanced by utilizing [Plugins](#) and leveraging [Function Calling](#). This allows agents to dynamically interact with external services or execute complex tasks, further expanding the scope and versatility of the AI system within diverse applications.

Learn how to configure agents to use plugins [here](#).

Agent Messages

Agent messaging, including both input and response, is built upon the core content types of the Semantic Kernel, providing a unified structure for communication. This design choice simplifies the process of transitioning from traditional chat-completion patterns to more advanced agent-driven patterns in your application development. By leveraging familiar Semantic Kernel content types, developers can seamlessly integrate agent capabilities into their applications without needing to overhaul existing systems. This streamlining ensures that as you evolve from basic

conversational AI to more autonomous, task-oriented agents, the underlying framework remains consistent, making development faster and more efficient.

💡 Tip

API reference:

- [ChatHistory](#)
- [ChatMessageContent](#)
- [KernelContent](#)
- [StreamingKernelContent](#)
- [FileReferenceContent](#)
- [AnnotationContent](#)

Templating

An agent's role is primarily shaped by the instructions it receives, which dictate its behavior and actions. Similar to invoking a `Kernel prompt`, an agent's instructions can include templated parameters—both values and functions—that are dynamically substituted during execution. This enables flexible, context-aware responses, allowing the agent to adjust its output based on real-time input.

Additionally, an agent can be configured directly using a Prompt Template Configuration, providing developers with a structured and reusable way to define its behavior. This approach offers a powerful tool for standardizing and customizing agent instructions, ensuring consistency across various use cases while still maintaining dynamic adaptability.

Learn more about how to create an agent with Semantic Kernel template [here](#).

Declarative Spec

The documentation on using declarative specs is coming soon.

Next steps

[Explore the Common Agent Invocation API](#)

Last updated on 05/29/2025